

Windows Event Log Detection Lab - Interview Explanation

Project Summary

This project is a Windows Event Log detection lab for a SOC Analyst portfolio.

In simple words, the lab uses sample Windows logs and Sigma-style YAML rules to detect suspicious activity. The Python script reads the logs, applies detection rules, maps findings to MITRE ATT&CK, and creates an analyst-ready report.

This is realistic because SOC analysts often investigate Windows Security logs in a SIEM. Common investigations include failed logons, brute-force attempts, suspicious PowerShell, and unexpected privilege changes.

What The Tool Does

The tool performs five main steps:

1. Reads sample Windows event logs from windows_events.jsonl.
2. Loads detection rules from the rules folder.
3. Applies the rules to find suspicious activity.
4. Adds MITRE ATT&CK tactic and technique context.
5. Generates a Markdown detection report for analyst review.

Interview version:

I built a Windows Event Log detection lab that simulates how a SOC analyst or detection engineer identifies suspicious Windows activity. It uses sample event logs, Sigma-style YAML rules, Python detection logic, MITRE ATT&CK mapping, and a final investigation report.

Why Each File Exists

data/windows_events.jsonl

This file contains sample Windows events. Each line is one event written as JSON.

The dataset includes:

- * Failed logons
- * Successful logons
- * Suspicious PowerShell process creation
- * PowerShell script block logging
- * Privileged group membership changes

- * Normal activity for comparison

Interview explanation:

I used JSON Lines to represent Windows event logs because it is easy to parse in Python and similar to how normalized logs might appear in a SIEM or log pipeline.

rules/brute_force.yml

This rule detects several failed logons from the same username and source IP followed by a successful logon.

Important Event IDs:

- * 4625: Failed logon
- * 4624: Successful logon

Interview explanation:

This rule looks for a pattern where an attacker may be guessing passwords and then successfully authenticates. That is stronger than a single failed logon because it shows repeated failures followed by possible account compromise.

rules/suspicious_powershell.yml

This rule detects suspicious PowerShell activity.

Important Event IDs:

- * 4688: Process creation
- * 4104: PowerShell script block logging

Suspicious keywords include:

- * 'EncodedCommand'
- * '-enc'
- * 'ExecutionPolicy Bypass'
- * 'DownloadString'
- * 'IEX'

Interview explanation:

PowerShell is used by administrators, but attackers also abuse it for execution, downloading payloads, and running encoded commands. This rule searches for suspicious

command-line patterns that may indicate malicious PowerShell usage.

rules/privilege_change.yml

This rule detects when a user is added to a privileged group.

Important Event IDs:

- * 4728: User added to a global security group
- * 4732: User added to a local security group

Privileged groups include:

- * Administrators
- * Domain Admins
- * Enterprise Admins

Interview explanation:

Privileged group changes are important because attackers may add accounts to admin groups for persistence or privilege escalation. This rule helps identify when access changes should be reviewed.

rules/failed_logon.yml

This rule detects individual failed logon events.

Important Event ID:

- * 4625: Failed logon

Interview explanation:

A single failed logon is not always malicious, but tracking failed logons helps analysts identify authentication anomalies and build context for brute-force investigations.

src/detect.py

This is the main Python script.

It:

- * loads sample event logs
- * loads YAML-style rules

- * runs detection logic
- * maps findings to MITRE ATT&CK
- * recommends analyst next steps
- * writes the final report

Interview explanation:

The Python script acts like a small detection engine. It reads logs and rules, identifies matching activity, and produces a structured report that a SOC analyst could review.

reports/detection_report.md

This is the generated detection report.

It includes:

- * total events analyzed
- * total rules loaded
- * number of findings
- * severity counts
- * finding details
- * MITRE ATT&CK mapping
- * recommended analyst actions

How Detection Logic Works

The lab uses four detection categories.

Failed Logon Detection

The script checks for Windows Event ID 4625.

This event means an account failed to log on.

Why it matters:

Failed logons can show password guessing, expired credentials, service account issues, or early signs of brute-force activity.

Brute Force Detection

The script groups events by:

- * username
- * source IP

Then it checks whether there are at least five failed logons followed by a successful logon.

Why it matters:

Multiple failures followed by success may indicate that an attacker guessed the correct password or used valid credentials after repeated attempts.

Suspicious PowerShell Detection

The script checks process creation and PowerShell script block events for risky keywords.

Examples:

- * encoded commands
- * execution policy bypass
- * download cradle behavior
- * 'IEX'

Why it matters:

Suspicious PowerShell can indicate execution of malicious commands, payload download, or attacker automation.

Privilege Change Detection

The script checks whether users were added to privileged groups.

Why it matters:

Unexpected admin group membership changes can indicate privilege escalation, persistence, or unauthorized access.

How MITRE ATT&CK Mapping Works

Each rule includes MITRE ATT&CK context.

Examples:

- * Brute Force maps to Credential Access and T1110 Brute Force.

- * Suspicious PowerShell maps to Execution and T1059.001 PowerShell.
- * Privilege Change maps to Persistence and T1098 Account Manipulation.

Interview explanation:

I mapped each detection rule to MITRE ATT&CK so the findings describe attacker behavior, not just raw event IDs. This helps analysts understand the stage and technique of the activity.

How To Describe It On Your Resume

Resume bullet:

text

Developed Windows event log detection rules to identify brute-force activity, suspicious PowerShell usage, privilege changes, and authentication anomalies.

Stronger version:

text

Built a Python-based Windows Event Log detection lab using Sigma-style YAML rules to detect brute-force behavior, suspicious PowerShell execution, failed logons, and privileged account changes mapped to MITRE ATT&CK.

How To Describe It On GitHub

GitHub description:

text

Windows Event Log detection lab using Python and Sigma-style YAML rules to identify brute force, suspicious PowerShell, failed logons, and privileged account changes.

GitHub tags/topics:

text

windows-event-logs

soc

cybersecurity

python

sigma

blue-team

mitre-attack
detection-engineering
incident-response
siem

Interview Talk Track

If an interviewer asks, "Tell me about this project," say:

I built a Windows Event Log detection lab that uses sample Windows logs and Sigma-style rules. The lab detects failed logons, brute-force behavior, suspicious PowerShell execution, and privileged group membership changes. The Python script reads the logs and rules, applies detection logic, maps findings to MITRE ATT&CK, and generates an investigation report.

If they ask about Windows event IDs, say:

I used Event ID 4625 for failed logons, 4624 for successful logons, 4688 for process creation, 4104 for PowerShell script block logging, and 4728/4732 for privileged group membership changes.

If they ask why this matters in a SOC, say:

These detections represent common SOC investigations. Brute force can indicate credential attacks, suspicious PowerShell can indicate malicious execution, and privilege changes can indicate persistence or privilege escalation.

Final strong sentence:

This project shows I understand Windows security logs, detection logic, MITRE ATT&CK mapping, and how to document findings in a SOC investigation workflow.